FE515 Lecture 3: Live Market Data and Option Liquidity in R

2026-09-17

Contents

1 Class goal **1**

2 Install and load `quantmod` **2**

3 Download equity data directly into R **2**

4 Brief review of last lecture **4**

5 Download a live option chain **6**

6 Inspect one expiration date **6**

7 Measure the bid-ask spread **8**

8 ITM versus OTM liquidity **10**

9 Visualize option liquidity across strikes **11**

10 Key takeaways **13**

1 Class goal

In the previous lecture, we worked with financial data that had already been saved in a file. Today we will extend that workflow by downloading market data directly into R and then using a live option chain to study liquidity.

The main question for today is:

> **How can we use R to download market data directly and identify where an option market is most liquid?**

By the end of the walkthrough, we will be able to:

- install and load the `quantmod` package;
- download historical equity prices directly into R;
- reproduce a short version of last lecture's return analysis;
- download and inspect an option chain;
- interpret bid, ask, volume, open interest, and moneyness;
- calculate absolute and relative bid-ask spreads;
- identify contracts with the tightest spreads;
- compare liquidity for in-the-money and out-of-the-money options.

2 Install and load `quantmod`

R includes many built-in functions, but specialized packages add additional tools. For this lecture we will use `quantmod`, which contains functions for downloading and working with financial market data.

You only need to **install** a package once on a computer. You need to **load** the package each time you start a new R session.

```
# Run this once if quantmod is not already installed:
#install.packages("quantmod")

library(quantmod)
```

The main functions we will use today are:

- `getSymbols()` for historical market data;
- `getQuote()` for a current quote;
- `getOptionChain()` for option-chain data.

3 Download equity data directly into R

Suppose we want to study Apple stock. Instead of downloading a CSV file manually, we can ask R to retrieve the data for us.

```
ticker <- "AAPL"
start_date <- "2024-01-01"

getSymbols(
  Symbols = ticker,
  from = start_date,
  to = Sys.Date()
)
```

```
## [1] "AAPL"
```

By default, `getSymbols()` creates an object with the same name as the ticker. Since we downloaded Apple, R created an object called `AAPL`.

```
head(AAPL)
```

```
##            AAPL.Open AAPL.High AAPL.Low AAPL.Close AAPL.Volume AAPL.Adjusted
## 2024-01-02    187.15    188.44   183.89     185.64    82488700      183.4040
## 2024-01-03    184.22    185.88   183.43     184.25    58414500      182.0307
## 2024-01-04    182.15    183.09   180.88     181.91    71983600      179.7189
## 2024-01-05    181.99    182.76   180.17     181.18    62379700      178.9977
## 2024-01-08    182.09    185.60   181.50     185.56    59144500      183.3250
## 2024-01-09    183.92    185.15   182.73     185.14    42841800      182.9100
```

```
tail(AAPL)
```

```
##            AAPL.Open AAPL.High AAPL.Low AAPL.Close AAPL.Volume AAPL.Adjusted
## 2026-09-09    315.49    319.15   309.90     315.34    65640000        315.34
## 2026-09-10    316.67    326.74   316.51     326.57    70011900        326.57
## 2026-09-11    327.45    336.22   326.30     332.27    50716900        332.27
## 2026-09-14    334.79    335.50   331.34     333.08    39269100        333.08
## 2026-09-15    330.14    331.78   328.35     331.34    31748200        331.34
## 2026-09-16    332.53    335.48   330.70     332.41    35920400        332.41
```

The columns contain:

- **Open**: first traded price of the day;
- **High**: highest traded price of the day;
- **Low**: lowest traded price of the day;
- **Close**: final traded price of the day;
- **Volume**: number of shares traded;
- **Adjusted**: closing price adjusted for events such as stock splits and dividends.

For historical return calculations, the adjusted price is generally the most useful column because it better represents the return earned by an investor.

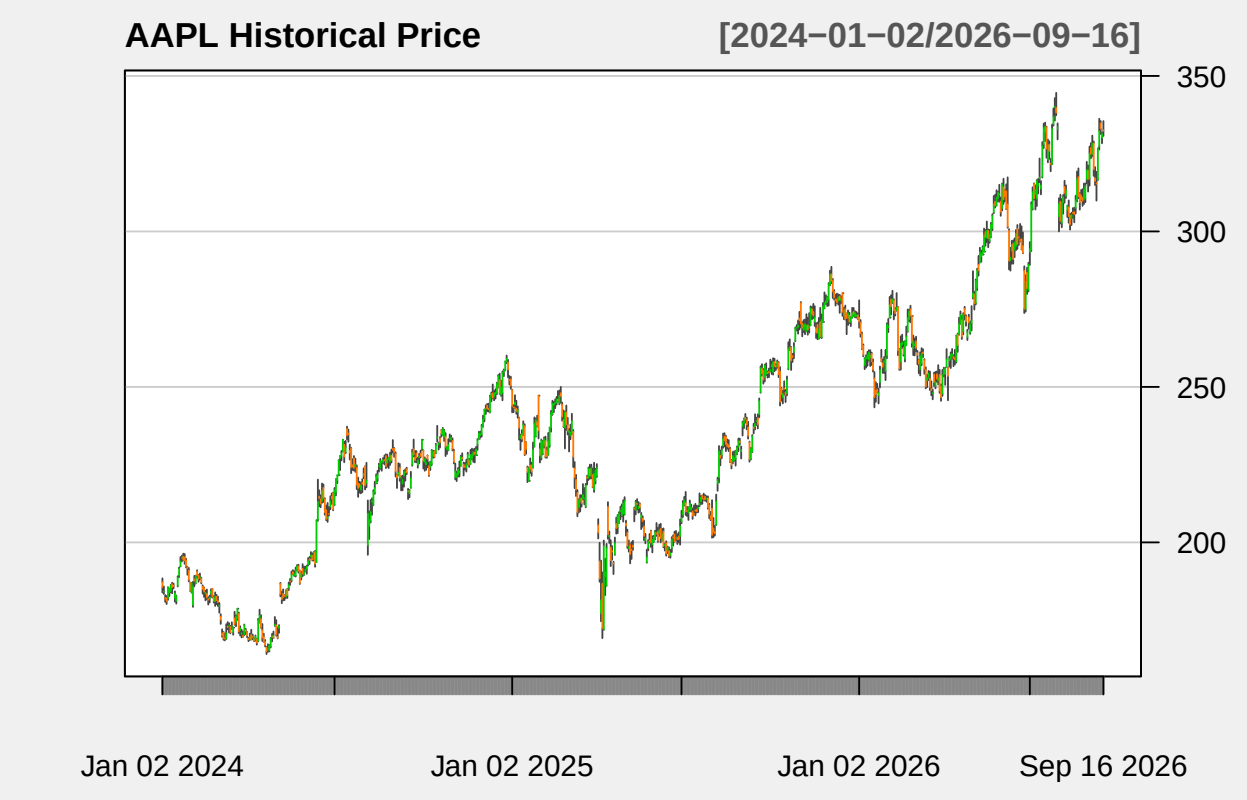
We can extract it with `Ad()`.

```
aapl_price <- Ad(AAPL)
head(aapl_price)
```

```
##            AAPL.Adjusted
## 2024-01-02      183.4040
## 2024-01-03      182.0307
## 2024-01-04      179.7189
## 2024-01-05      178.9977
## 2024-01-08      183.3250
## 2024-01-09      182.9100
```

We can also quickly visualize the downloaded data.

```
chartSeries(
  AAPL,
  theme = chartTheme("white"),
  TA = NULL,
  name = "AAPL Historical Price"
)
```



4 Brief review of last lecture

Last lecture, once we had a price series, we calculated returns and summarized performance. We can now repeat the same analysis using data downloaded directly from the market.

A simple return from day $t-1$ to day t is

$$R_t = \frac{P_t}{P_{t-1}} - 1.$$

In R:

```
aapl_returns <- aapl_price / lag(aapl_price) - 1
aapl_returns <- na.omit(aapl_returns)
```

```r
head(aapl_returns)
```

```
##            AAPL.Adjusted
## 2024-01-03  -0.007487543
## 2024-01-04  -0.012700042
## 2024-01-05  -0.004013139
## 2024-01-08   0.024175005
## 2024-01-09  -0.002263453
## 2024-01-10   0.005671387
```

We can summarize the daily return series.

```r
mean_daily_return <- mean(aapl_returns)
daily_volatility <- sd(aapl_returns)

mean_daily_return
```

```
## [1] 0.001028226
```

```r
daily_volatility
```

```
## [1] 0.01743013
```

If we use approximately 252 trading days per year,

$$\text{Annualized Mean Return} \approx 252 \bar{R}_{daily},$$

and

$$\text{Annualized Volatility} \approx \sqrt{252}\, \sigma_{daily}.$$

```r
annualized_return <- mean_daily_return * 252
annualized_volatility <- daily_volatility * sqrt(252)

annualized_return
```

```
## [1] 0.2591129
```

```r
annualized_volatility
```

```
## [1] 0.2766947
```

This should look familiar. The important new idea is that **R can now obtain the market data and analyze it in the same workflow**.

From here, we move to a new type of market data.

5 Download a live option chain

An option is a contract whose value depends on an underlying asset.

A **call option** gives the holder the right, but not the obligation, to buy the underlying asset at a fixed strike price.

A **put option** gives the holder the right, but not the obligation, to sell the underlying asset at a fixed strike price.

We will download Apple's option chain.

```
option_symbol <- "AAPL"

option_data <- tryCatch(
  getOptionChain(option_symbol, NULL),
  error = function(e) {
    message("Option-chain download failed. Try again later or use another ticker.")
    return(NULL)
  }
)
```

If the download succeeds, `option_data` contains the available expiration dates.

```
if (!is.null(option_data)) {
  names(option_data)
}
```

```
##  [1] "Sep.18.2026" "Sep.21.2026" "Sep.23.2026" "Sep.25.2026" "Oct.02.2026"
##  [6] "Oct.09.2026" "Oct.16.2026" "Oct.23.2026" "Oct.30.2026" "Nov.20.2026"
## [11] "Dec.18.2026" "Jan.15.2027" "Feb.19.2027" "Mar.19.2027" "Apr.16.2027"
## [16] "Jun.17.2027" "Sep.17.2027" "Dec.17.2027" "Jan.21.2028" "Mar.17.2028"
## [21] "Dec.15.2028"
```

This is our first important structural difference from the equity data. An option chain contains several layers:

$$\text{Ticker} \rightarrow \text{Expiration Date} \rightarrow \{\text{Calls}, \text{Puts}\} \rightarrow \text{Individual Contracts}.$$

6 Inspect one expiration date

Very short-dated options can sometimes have unusual spreads or very little time remaining. For this walkthrough, we will select the first expiration that is at least thirty days away when possible.

```r
if (!is.null(option_data)) {
  expiration_names <- names(option_data)
  expiration_dates <- as.Date(expiration_names, format = "%b.%d.%Y")

  valid_expirations <- which(expiration_dates >= Sys.Date() + 30)

  if (length(valid_expirations) > 0) {
    selected_index <- valid_expirations[1]
  } else {
    selected_index <- 1
  }

  selected_expiration <- expiration_names[selected_index]
  selected_expiration

  chain <- option_data[[selected_expiration]]
  names(chain)
}
```

```
## [1] "calls" "puts"
```

Each expiration generally contains one data frame for calls and one for puts.

```r
if (!is.null(option_data)) {
  head(chain$calls)
  head(chain$puts)
}
```

```
##            ContractID ConractSize Currency Expiration Strike Last Chg ChgPct
## 1 AAPL261023P00160000     REGULAR      USD 2026-10-23    160 0.08   0      0
## 2 AAPL261023P00165000     REGULAR      USD 2026-10-23    165 0.11   0      0
## 3 AAPL261023P00195000     REGULAR      USD 2026-10-23    195 0.07   0      0
## 4 AAPL261023P00200000     REGULAR      USD 2026-10-23    200 0.17   0      0
## 5 AAPL261023P00205000     REGULAR      USD 2026-10-23    205 0.10   0      0
## 6 AAPL261023P00215000     REGULAR      USD 2026-10-23    215 0.12   0      0
##   Bid  Ask Vol OI       LastTradeTime        IV   ITM
## 1  NA 0.06  27 27 2026-09-15 15:43:12 0.8554702 FALSE
## 2  NA 0.50   1  1 2026-09-15 12:23:41 1.0620164 FALSE
## 3   0 0.41   1  2 2026-09-09 10:44:29 0.7373073 FALSE
## 4   0 0.38   2  2 2026-09-04 11:55:41 0.6992218 FALSE
## 5   0 0.34   2  2 2026-09-04 11:55:56 0.6601596 FALSE
## 6   0 0.10   5  6 2026-09-15 11:15:27 0.5234423 FALSE
```

Some of the most useful columns are:

- **Strike**: exercise price of the option;

- **Bid**: highest price currently offered by a buyer;
- **Ask**: lowest price currently offered by a seller;
- **Vol**: recent trading volume;
- **OI**: open interest, or the number of currently outstanding contracts;
- **IV**: implied volatility;
- **ITM**: whether the contract is currently in the money.

Discussion question: Why might an option have a very different bid and ask even when both quotes refer to the same contract?

7 Measure the bid-ask spread

The bid-ask spread is one of the simplest measures of trading cost and market liquidity.

The **absolute spread** is

$$\text{Spread} = \text{Ask} - \text{Bid}.$$

However, comparing absolute spreads across contracts can be misleading. A $1 spread on a $100 option is very different from a $1 spread on a $2 option.

We therefore also calculate the midpoint

$$\text{Mid} = \frac{\text{Bid} + \text{Ask}}{2}$$

and the **relative spread**

$$\text{Relative Spread} = \frac{\text{Ask} - \text{Bid}}{\text{Mid}}.$$

A smaller relative spread generally indicates a tighter, more liquid market.

```
if (!is.null(option_data)) {
  calls <- chain$calls

  calls$Mid <- 0.5 * (calls$Bid + calls$Ask)
  calls$Spread <- calls$Ask - calls$Bid
  calls$RelativeSpread <- calls$Spread / calls$Mid

  head(calls)
}
```

```
##            ContractID ConractSize Currency Expiration Strike   Last Chg ChgPct
## 1 AAPL261023C00110000     REGULAR      USD 2026-10-23    110 206.60   0      0
## 2 AAPL261023C00145000     REGULAR      USD 2026-10-23    145 176.00   0      0
## 3 AAPL261023C00210000     REGULAR      USD 2026-10-23    210 110.97   0      0
```

```
## 4 AAPL261023C00215000     REGULAR      USD 2026-10-23    215 103.85   0      0
## 5 AAPL261023C00220000     REGULAR      USD 2026-10-23    220 101.28   0      0
## 6 AAPL261023C00225000     REGULAR      USD 2026-10-23    225 109.90   0      0
##      Bid    Ask Vol OI       LastTradeTime        IV  ITM     Mid Spread
## 1 224.40 227.85  NA  1 2026-09-08 12:28:09 1.9111333 TRUE 226.125   3.45
## 2 189.60 193.00  NA  1 2026-09-10 09:41:03 1.4997584 TRUE 191.300   3.40
## 3 125.35 128.60  NA  1 2026-09-10 09:31:20 0.7534204 TRUE 126.975   3.25
## 4 119.75 123.35  NA  2 2026-09-08 10:13:49 0.6015665 TRUE 121.550   3.60
## 5 115.10 118.40   4  2 2026-09-10 10:32:17 0.6386755 TRUE 116.750   3.30
## 6 110.55 113.75  NA  6 2026-09-11 14:13:27 0.6857942 TRUE 112.150   3.20
##   RelativeSpread
## 1     0.01525705
## 2     0.01777313
## 3     0.02559559
## 4     0.02961744
## 5     0.02826552
## 6     0.02853321
```

Some contracts may not have a usable two-sided quote. For this analysis, we will keep contracts with positive bid and ask prices and a finite relative spread.

```
if (!is.null(option_data)) {
  calls_clean <- calls[
    calls$Bid > 0 &
    calls$Ask > 0 &
    is.finite(calls$RelativeSpread),
  ]

  head(calls_clean)
}
```

```
##            ContractID ConractSize Currency Expiration Strike   Last Chg ChgPct
## 1 AAPL261023C00110000     REGULAR      USD 2026-10-23    110 206.60   0      0
## 2 AAPL261023C00145000     REGULAR      USD 2026-10-23    145 176.00   0      0
## 3 AAPL261023C00210000     REGULAR      USD 2026-10-23    210 110.97   0      0
## 4 AAPL261023C00215000     REGULAR      USD 2026-10-23    215 103.85   0      0
## 5 AAPL261023C00220000     REGULAR      USD 2026-10-23    220 101.28   0      0
## 6 AAPL261023C00225000     REGULAR      USD 2026-10-23    225 109.90   0      0
##      Bid    Ask Vol OI       LastTradeTime        IV  ITM     Mid Spread
## 1 224.40 227.85  NA  1 2026-09-08 12:28:09 1.9111333 TRUE 226.125   3.45
## 2 189.60 193.00  NA  1 2026-09-10 09:41:03 1.4997584 TRUE 191.300   3.40
## 3 125.35 128.60  NA  1 2026-09-10 09:31:20 0.7534204 TRUE 126.975   3.25
## 4 119.75 123.35  NA  2 2026-09-08 10:13:49 0.6015665 TRUE 121.550   3.60
## 5 115.10 118.40   4  2 2026-09-10 10:32:17 0.6386755 TRUE 116.750   3.30
## 6 110.55 113.75  NA  6 2026-09-11 14:13:27 0.6857942 TRUE 112.150   3.20
##   RelativeSpread
## 1     0.01525705
```

```
## 2     0.01777313
## 3     0.02559559
## 4     0.02961744
## 5     0.02826552
## 6     0.02853321
```

Now we can identify the contract with the tightest relative spread.

```
if (!is.null(option_data) && nrow(calls_clean) > 0) {
  tightest_index <- which.min(calls_clean$RelativeSpread)

  columns_to_show <- intersect(
    c("Strike", "Bid", "Ask", "Mid", "Spread", "RelativeSpread", "Vol", "OI", "IV", "ITM"),
    names(calls_clean)
  )

  calls_clean[tightest_index, columns_to_show, drop = FALSE]
}
```

```
##   Strike   Bid    Ask     Mid Spread RelativeSpread Vol OI       IV  ITM
## 1    110 224.4 227.85 226.125   3.45     0.01525705  NA  1 1.911133 TRUE
```

8 ITM versus OTM liquidity

To classify call options as in the money or out of the money, we first need the current stock price.

```
quote_data <- tryCatch(
  getQuote(option_symbol),
  error = function(e) NULL
)

if (!is.null(quote_data)) {
  spot <- as.numeric(quote_data$Last[1])
} else {
  # Fallback if the live quote request fails
  spot <- as.numeric(last(Cl(AAPL)))
}

spot
```

```
## [1] 336.38
```

For a call option:

$$K < S \Rightarrow \text{ITM},$$

$$K > S \Rightarrow \text{OTM},$$

where K is the strike price and S is the current stock price.

We will create our own moneyness label.

```
if (!is.null(option_data) && nrow(calls_clean) > 0) {
  calls_clean$Moneyness <- ifelse(
    calls_clean$Strike < spot,
    "ITM",
    ifelse(calls_clean$Strike > spot, "OTM", "ATM")
  )

  table(calls_clean$Moneyness)
}
```

```
##
## ITM OTM
##  28  14
```

Now compare the typical relative spread across moneyness groups. We use the **median** because option spreads can contain very large outliers.

```
if (!is.null(option_data) && nrow(calls_clean) > 0) {
  spread_by_moneyness <- aggregate(
    RelativeSpread ~ Moneyness,
    data = calls_clean,
    FUN = median,
    na.rm = TRUE
  )

  spread_by_moneyness
}
```

```
##   Moneyness RelativeSpread
## 1       ITM     0.03530767
## 2       OTM     0.10783412
```

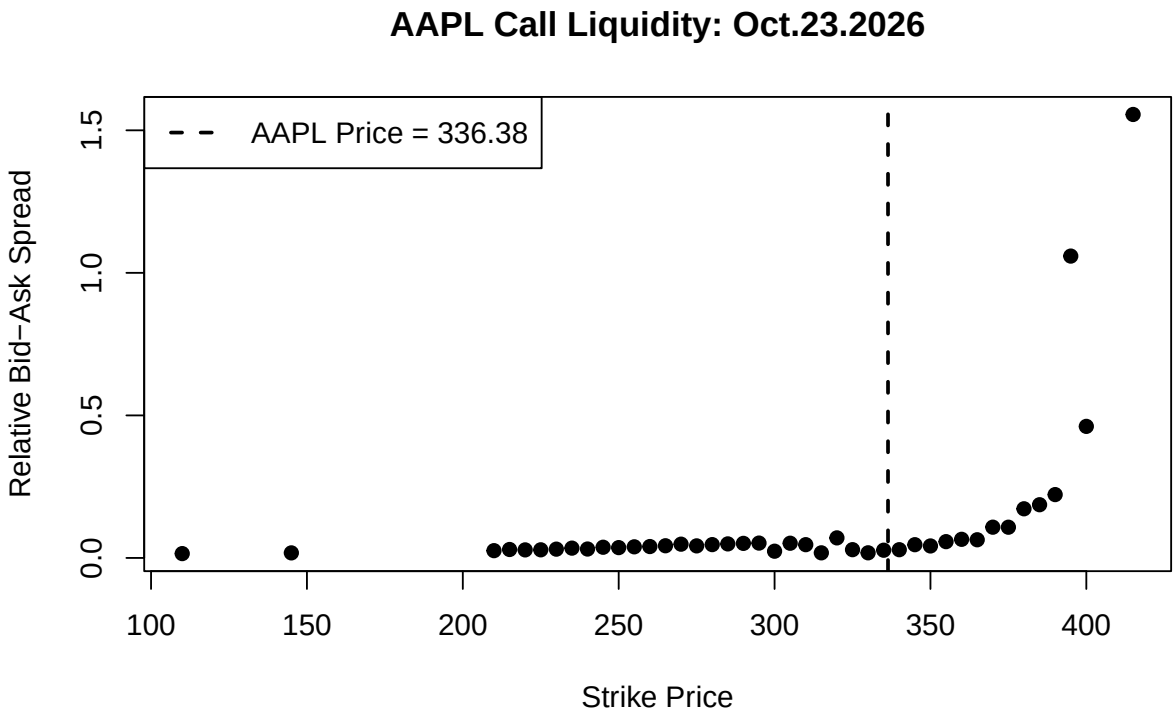
9 Visualize option liquidity across strikes

A table tells us which contract has the smallest spread, but a graph makes it easier to see how liquidity changes across the entire option chain.

```r
if (!is.null(option_data) && nrow(calls_clean) > 0) {
  plot(
    calls_clean$Strike,
    calls_clean$RelativeSpread,
    xlab = "Strike Price",
    ylab = "Relative Bid-Ask Spread",
    main = paste("AAPL Call Liquidity:", selected_expiration),
    pch = 19
  )

  abline(v = spot, lty = 2, lwd = 2)

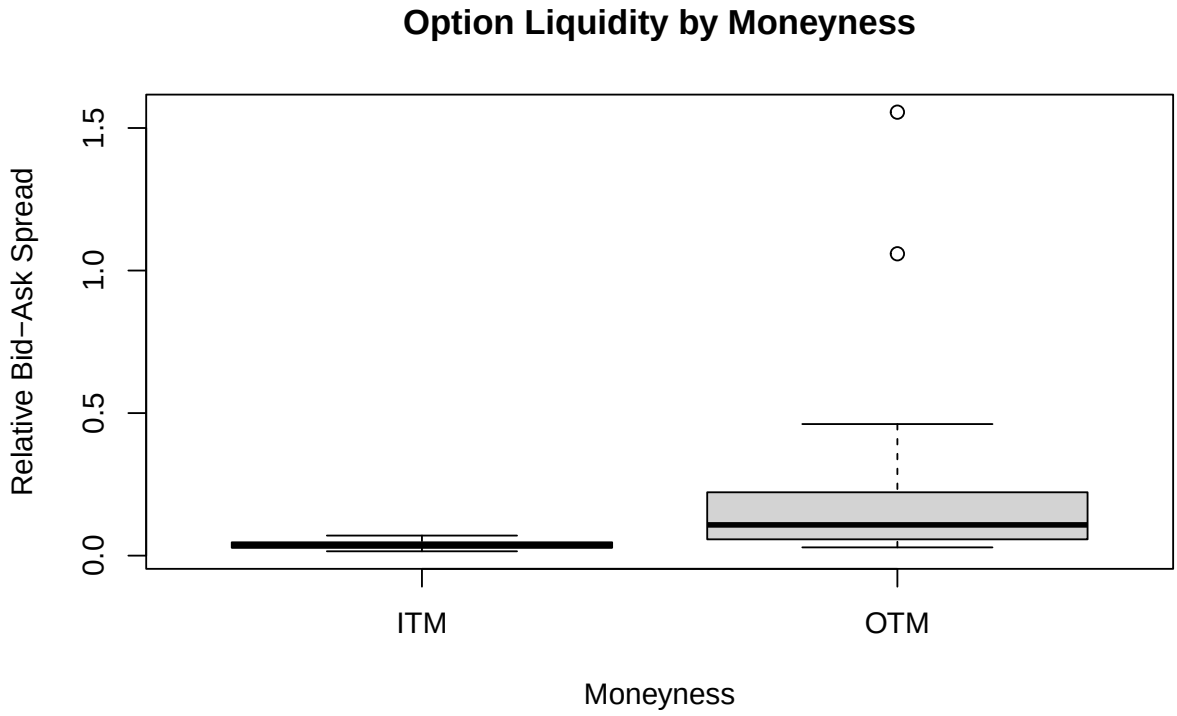
  legend(
    "topleft",
    legend = paste("AAPL Price =", round(spot, 2)),
    lty = 2,
    lwd = 2
  )
}
```



The dashed vertical line marks the current stock price. Strikes to the left are generally ITM calls and strikes to the right are generally OTM calls.

We can also compare the distribution of relative spreads directly.

```r
if (!is.null(option_data) && nrow(calls_clean) > 0) {
  boxplot(
    RelativeSpread ~ Moneyness,
    data = calls_clean,
    xlab = "Moneyness",
    ylab = "Relative Bid-Ask Spread",
    main = "Option Liquidity by Moneyness"
  )
}
```



10 Key takeaways

By the end of this walkthrough, we have used R to move through a complete market-data workflow:

1. Load a financial-data package.
2. Download equity prices directly into R.
3. Reproduce familiar return calculations without importing a CSV file.
4. Download a live option chain.
5. Inspect nested market-data structures.
6. Calculate bid-ask midpoints and spreads.
7. Compare liquidity using spread.
8. Classify options by moneyness.

9. Compare ITM and OTM option liquidity.
10. Visualize where option-market liquidity is concentrated.

The main lesson is that R is not only a tool for analyzing a dataset after it has been prepared. It can also be used to **retrieve, inspect, clean, and analyze live financial market data in one workflow**.